LABORATORIO #1 • POO + N8N • ESCENARIO 1

Central de facturación con días hábiles

Diseño, lógica de negocio y prueba de la integración

Nombre del estudiante: _____ Fecha: _____

1. Problema y solución

Una empresa emite facturas que vencen quince días hábiles después de la fecha de emisión. Un cálculo ingenuo de días naturales puede incluir sábados, domingos o feriados, y marcar facturas como vencidas antes de tiempo. La solución automatiza el cálculo y actualiza los registros verificables de JSON Server.

La integración utiliza Postman como cliente HTTP; n8n como orquestador; JSON Server como persistencia; Nager.Date (Public Holiday API) como calendario externo; y JavaScript orientado a objetos para aplicar las reglas. Todo se ejecuta sin una interfaz web adicional.

2. Arquitectura y decisiones

Módulo	Responsabilidad
Webhook POST n8n	Recibe una factura completa sin ID, o un facturaId existente.
JSON Server (puerto 3000)	POST crea; GET consulta; PATCH persiste vencimiento/estado.
Nager.Date	GET feriados públicos de Costa Rica del año de emisión y siguiente.
Code + POO	Valida, crea Factura/CalendarioLaboral y calcula fecha y estado.
IF + Respond to Webhook	Separa rutas de éxito/error y responde JSON a Postman.

La fecha se interpreta como fecha civil YYYY-MM-DD con operaciones UTC para no saltar un día por diferencias horarias. La fecha de revisión se calcula con America/Costa_Rica. Se consulta además el año posterior a la emisión para resolver plazos que crucen de diciembre a enero.

3. Recorrido de datos

Postman → Webhook → Validación → IF nueva/existente → POST/GET JSON Server → Validar registro → GET Nager.Date (dos años) → Code POO → IF actualización necesaria → PATCH o sin cambios → respuesta JSON.

El usuario no necesita asignar un ID cuando crea la factura: JSON Server lo genera y n8n lo devuelve. La colección de Postman guarda ese ID para solicitudes posteriores.

Reglas de negocio y modelo POO

4. Clases y responsabilidades

Factura.js encapsula los datos con un campo privado #datos, valida id, cliente, monto, estado y fecha de emisión, calcula vencimiento, decide si ya venció y planifica el PATCH. CalendarioLaboral.js encapsula los feriados en #feriados y los años verificados en #aniosDisponibles; decide si una fecha es hábil y suma los días permitidos. Las dos clases pueden ejecutarse sin n8n usando main.js.

5. Reglas implementadas

Regla	Comportamiento verificable
R1: validez	Cliente no vacío, monto positivo, fecha ISO real, estado admitido e ID al persistir.
R2: plazo	Se cuentan 15 días hábiles desde el día posterior a la emisión.
R3: calendario	Sábados, domingos y feriados nacionales públicos (global = true) no cuentan.
R4: mora	Solo pasa a vencida cuando hoy (Costa Rica) es posterior al vencimiento.
R5: conservación	Pagada/anulada conserva su estado y no ejecuta PATCH.
R6: idempotencia	No se repite PATCH cuando fecha y estado ya son correctos.
R7: errores	Si faltan datos o feriados verificados, se detiene el cálculo.

6. Compartir reglas entre JavaScript y n8n

La implementación única vive en src/Factura.js, src/CalendarioLaboral.js y src/fechas.js. El script scripts/generar-integracion.mjs incrusta esas clases, sin imports de Node, en el nodo “POO - Calcular 15 dias habiles” del workflow exportable. De esta forma no se mantiene por separado otro algoritmo, lo que reduce inconsistencias al corregir reglas.

Ejemplo: factura emitida el 19/10/2026 con 15 días hábiles. Usando el calendario simulado de pruebas, su vencimiento es el 09/11/2026. En producción, n8n debe calcular usando la respuesta real vigente de Nager.Date, no las muestras del repositorio.

7. Tipos de error previstos

- HTTP 400: JSON incompleto, cliente incorrecto, monto no positivo o fecha inválida; no se inserta factura.
- HTTP 404: el facturaId solicitado no existe en JSON Server.
- HTTP 422: factura guardada tiene datos inválidos; no se hace PATCH.
- HTTP 502/503: respuesta inválida, servidor inaccesible o API externa caída; se informa explícitamente.

Operación, pruebas y defensa

8. Demostración paso a paso

1. Arrancar JSON Server y n8n: INICIAR_TODO_WINDOWS.bat o INICIAR_CON_DOCKER_WINDOWS.bat. Usar el JSON de workflow correspondiente a la red elegida.

2. Abrir http://localhost:5678. Importar el workflow e iniciar “Listen for Test Event” en el Webhook.

3. Importar la colección de Postman y enviar la solicitud “01 - CREAR Y PROCESAR SIN ID”. El Body contiene cliente, monto, moneda y fechaEmision, sin un ID preasignado.

4. Inspeccionar la respuesta (facturaId, fechaVencimiento, estado). Confirmar el registro en http://localhost:3000/facturas y revisar las ramas IF ejecutadas.

5. Demostrar error 400 con una solicitud sin monto, error 404 con un facturaId inexistente, y conservación del estado de una factura pagada.

6. Publicar el workflow antes de usar la URL estable /webhook/. En modo prueba se usa /webhook-test/ y n8n debe estar escuchando.

9. Pruebas realizadas y alcance

El proyecto incluye 22 pruebas automáticas: reglas POO independientes; equivalencia de la clase con código inyectado; estructura JSON y conexiones; sintaxis de todos los nodos Code; y simulación local de nuevos registros, existentes, errores 400/404/422, pago y caída de la API. Los datos simulados no constituyen una ejecución real de n8n ni una consulta en vivo de Nager.Date.

10. Limitaciones y decisiones técnicas

En Windows n8n y JSON Server deben compartir la misma máquina y usan 127.0.0.1:3000 internamente. En Docker se conectan por json-server:3000. Un n8n Cloud no puede resolver la dirección localhost de una PC. Si la API externa falla después de crear una factura nueva, esta puede quedar creada sin fecha calculada; se informa el ID para poder reintentar. No se implementan transacciones distribuidas ni reversiones automáticas.

11. Conclusión

La solución une POO y automatización en un proceso de negocio demostrable: entrada HTTP, lectura/escritura persistente, consulta externa, bifurcaciones y respuesta estructurada. La parte más delicada es mantener un calendario válido para cada año implicado y no confundir “factura creada” con “factura completamente procesada”. Los tests protegen las reglas; la demostración en vivo confirma la integración real.